

GUÍA RESUMEN — Diseño de Sistemas

Resumen integral para diseñar sistemas a escala con Flutter + Supabase sin volver a leer el módulo completo. Hilo conductor: Estimación de escala → Modelado de datos → Cache → Realtime → CAP/Alta disponibilidad → Seguridad → Observabilidad → Offline → 4 casos integradores.

0 La Plantilla de 10 pasos

CUALQUIER DISEÑO, EN ORDEN

1 Alcance qué SÍ y qué NO	2 Escala QPS, storage, ratios	3 Datos entidades, índices, RLS	4 Arquitectura diagrama de cajas	5 Componentes API, DB, cache, CDN
6 Decisiones cada una con su trade-off	7 Consistencia CAP, SLA, patrón	8 Seguridad authn/authz, RLS	9 Observabilidad logs, métricas, alertas	10 Riesgos cuellos de botella futuros

Regla de oro: "Elijo X porque el caso es Y, y el trade-off aceptable es Z." Toda decisión sin trade-off explícito es una decisión incompleta.

Paso	Qué haces	Clave	Dónde aprender más
1. Alcance	Usuarios, funcionalidad, latencia, mobile/web, MVP vs maduro	Define también el fuera de scope	01-clarificar-requisitos.md
2. Escala	QPS lecturas/escrituras, storage, ratio L/E	Números antes de dibujar	02-estimacion-de-escala.md
3. Datos	Entidades, índices, RLS, SQL vs NoSQL	Índices para los path reales	04-modelado-de-datos.md
4. Arquitectura	Diagrama de cajas y flujos	App → CDN/API → Cache → DB	03-componentes-arquitectura.md
5. Componentes	Elegir cada pieza	Cache, realtime, CDN, Edge	03 · 05 · 06
6. Decisiones	Justificar con trade-offs	"X porque Y, trade-off Z"	11-plantilla-diseno-sistema.md
7. Consistencia	Declarar CAP y SLA	strong vs eventual por dato	07-escalabilidad-alta-disponibilidad.md
8. Seguridad	HTTPS, RLS, menor privilegio	Deny-by-default	08-seguridad-en-diseno.md
9. Observabilidad	Métricas + alertas accionables	Los 3 pilares	09-observabilidad-monitoreo.md
10. Riesgos	Cuellos de botella futuros y mitigación	Cerrar con "el siguiente sería X"	11-plantilla-diseno-sistema.md

NO hagas: × diseñar sin clarificar · × dibujar sin números · × decisiones sin trade-off · × olvidar CAP/seguridad/observabilidad · × terminar sin riesgos y evolución.

1 Estimación de Escala

BACK-OF-THE-ENVELOPE

Antes de dibujar el diagrama, calcula **qué tanto debe aguantar**. Estos números guían todas las demás decisiones (cache, replicas, sharding).

Las 4 fórmulas

Métrica	Fórmula	Ejemplo (feed)
QPS lecturas	<code>usuarios activos × acciones/usuario ÷ segundos</code>	10M MAU · 5% pico · 100 lecturas/día → ~1.7K QPS
QPS escrituras	<code>acciones/día ÷ 86,400</code>	1.5M posts+likes/día → ~17 QPS
Storage	<code>filas × bytes × retención</code>	Media del feed → ~2.7 TB/año
Ratio L/E	<code>lecturas ÷ escrituras</code>	~10:1 → sistema read-heavy

Lectura del ratio: ratio 10:1 → **read-heavy** → el cuello de botella son las lecturas → cache + read replicas. Ratio cercano a 1 o mayor en escrituras → **write-heavy** → el cuello es la DB → particionado/colas.

Latencia vs throughput: latencia = tiempo de una acción; throughput = acciones por segundo. Problema de latencia = "lento para uno". Problema de throughput = "rápido para uno, se hunde con muchos". Objetivo: *maximal throughput con latency aceptable*.

SQL vs NoSQL

Criterio	SQL (PostgreSQL/Supabase)	NoSQL
Esquema	Fijo, relacional	Flexible
Transacciones ACID	Sólidas	Variables
Relaciones/joins	Nativos	Se emulan en la app
Escalado	Vertical + réplicas; sharding manual	Horizontal por diseño
Cuándo	Casi siempre (empieza aquí)	Latencia extrema, KV, datos no estructurados

Regla práctica: empieza con RDBMS. Postgres (corazón de Supabase) cubre el 90% con JSONB, RLS y realtime.

Índices

Aceleran **lecturas** pero frenan **escrituras** y consumen espacio. Se crean para los **path de acceso reales**, no para todo.

```
CREATE INDEX idx_posts_user ON posts (user_id);
CREATE INDEX idx_posts_user_created ON posts (user_id, created_at DESC);
```

Replicación (más lecturas + tolerancia)

Tipo	Quién escribe	Trade-off
Master-Slave	Solo el master; slaves leen	Escrituras pueden tener lag (eventual consistency)
Master-Master	Ambos escriben	Conflictos entre masters — complejo, evitar al inicio

Sharding (último recurso)

Dividir la tabla en N bases por un **shard key** (ej. `Hash(user_id) % N`). Escala escrituras/storage pero rompe joins y complica rebalanceo. **Orden:** índices → cache → réplicas → federación → sharding.

Denormalización

Duplicar data a propósito para **leer sin joins**. Leer es rápido, pero cada escritura debe mantener las copias sincronizadas (riesgo de inconsistencia).

	Normalizado	Denormalizado
Escritura	Simple (1 lugar)	Compleja (mantener copias)
Lectura	Joins caros	Rápida (todo en la fila)

Árbol de decisión: ¿consulta lenta? → índices + `EXPLAIN` → ¿lectura repetitiva? → cache → ¿mucha concurrencia? → read replicas → ¿escrituras el cuello? → particionado → ¿nada funciona? → sharding.

Dónde cachear (de cerca a lejos del usuario)

App

Isar local, HTTP cache

CDN

media, estáticos

Server

respuestas HTTP/Edge

DB

query cache, views

Las 4 estrategias de actualización

Estrategia	Qué hace	Trade-off
Cache-aside (lazy)	Miss → lee DB → guarda en cache	Primer acceso lento; cachea solo lo usado recomendada
Write-through	Escribe cache + DB juntos	Lecturas siempre frescas; escrituras más lentas
Write-behind	Escribe cache, persiste async	Escrituras rapidísimas; riesgo de pérdida
Refresh-ahead	Refresca antes de expirar	Menos misses; puede predecir mal

TTL: tiempo de vida — mientras no expire el dato puede estar **stale**. **Invalidación:** cuando el dato cambia, hay que actualizar el cache (o el usuario ve data vieja). **Eviction:** LRU (Least Recently Used) es lo estándar.

Cache stampede (miss storm): si todos los caches expiran a la vez, miles pegan a la DB. Mitigación: jitter en TTL + stale-while-revalidate.

```
// Feed del repo (decisión completa):
Lectura: app (Isar) → si miss → API → Redis → si miss → Postgres → cachea → devuelve
Escritura: crear post → invalida el cache del feed del autor (write-through)
Justificación: ratio 10:1 read-heavy → cache-aside + invalidación al escribir.
```

Las 4 opciones

Técnica	Dirección	Cuándo
Polling	Cliente → servidor	Segundos de latencia, simple (refrescar cada 30s)
Long polling	Cliente mantiene la conexión hasta respuesta	Sin WebSocket, menos peticiones
WebSocket	Bidireccional full-duplex	Realtime de verdad (chat, presencia)
SSE	Servidor → cliente (unidireccional)	Updates solo del servidor

Asincronía (message/task queues)

Desacoplar el productor del consumidor: el mensaje **espera** si el consumidor cae. En Supabase: trigger en DB → Edge Function → notificación. **Cuidado:** si el consumidor va más lento, la cola crece (backpressure) — monitorear su tamaño.

Supabase Realtime (gestionado)

3 canales oficiales: **Postgres Changes** (recibe inserts/updates/deletes), **Broadcast** (mensajes entre clientes), **Presence** (quién está online).

Postgres —(WAL)—▶ Realtime —(WebSocket)—▶ Flutter App

```
supabase.channel('messages:room-1')
  .onPostgresChanges(event: insert, table: 'messages',
    callback: (payload) => emit(Message.fromJson(payload.newRecord)))
  .subscribe();
```

Árbol de decisión: ¿<1-2s? → polling · ¿unidireccional? → SSE · ¿bidireccional baja latencia? → **WebSocket (Supabase Realtime)** · ¿algo intermedio? → long polling.

Performance vs Scalability

Performance: lento para un usuario. **Scalability:** rápido para uno, lento bajo carga. Escalar: primero **vertical** (más CPU/RAM, simple), luego **horizontal** (más máquinas, con load balancing).

CAP theorem (elegir 2 de 3)

Consistency · Availability · Partition tolerance. La red no es confiable → siempre hay particiones → eliges entre C y A.

	CP (consistencia)	AP (disponibilidad)
Ante partición	Espera/bloquea hasta estar consistente	Responde con lo disponible (puede estar viejo)
Ideal para	Pagos, inventario, contadores	Feeds, chats, notificaciones

Patrones de consistencia

Patrón	Qué garantiza
Weak	Los reads pueden no ver el write (best-effort)
Eventual	Los reads lo verán eventualmente (ms) — DNS, feeds, likes
Strong	Todos los reads ven el write — RDBMS

Disponibilidad en números

Disponibilidad	Downtime/año	Downtime/día
99%	3d 15h	14m 24s
99.9%	8h 45m	1m 26s
99.99%	52m 35s	8.6s

Failover: active-passive (standby, más barato) vs active-active (ambos atienden, riesgo de conflictos). **Serie vs paralelo:** componentes en serie *multiplican* la indisponibilidad; redundancia en paralelo la reduce.

En Supabase no reinventes la HA: Auth, DB, Storage y Realtime tienen SLA gestionado. Tu trabajo es: no crear puntos únicos de fallo en tu código, elegir el SLA correcto para el caso y tener cache local (offline) como plan B.

Capas (ninguna basta sola)

App (secrets nunca en el bundle) → **HTTPS/TLS** → API (Auth + RLS) → DB (menor privilegio + backups cifrados).

Authn vs Authz

Authentication: quién eres (login, OAuth, magic link). **Authorization:** qué puedes hacer (permisos por fila). Error típico: validar solo el login y olvidar la autorización.

RLS como capa de autorización

El módulo 03 lo implementa; aquí la visión de diseño: RLS se evalúa **en la DB** (no en la app). Aunque roben un JWT, la policy sigue protegiendo. Cada policy usa `auth.uid()` (el `sub` del JWT de Supabase).

```
CREATE POLICY "read_own_drafts" ON posts
FOR SELECT USING (auth.uid() = user_id AND status = 'draft');
```

Principio de menor privilegio

Quién	Permisos correctos
Anon key en la app	Solo lo público (RLS niega el resto)
Service role key	Nunca en la app; solo Edge Functions/servidor
Edge Functions	Solo las operaciones que necesitan
Storage buckets	Bucket público vs privado + RLS en storage.objects

Checklist mínimo: HTTPS en todo · tokens en `flutter_secure_storage` · RLS en todas las tablas · service role solo en backend · `.env` en `.gitignore` · menor privilegio en roles.

7 Observabilidad y Resiliencia

Los 3 pilares

Pilar	Qué es	Pregunta que responde
Logs	Eventos discretos (JSON, con trace_id)	¿Qué pasó?
Métricas	Números agregados (USE: Utilización, Saturación, Errores)	¿Cuánto / con qué frecuencia?
Tracing	Recorrido del request por todos los componentes	¿Dónde se tardó?

Alertas accionables (4 pasos)

1) Métrico correcto (error rate, latencia p95) · 2) Umbral realista · 3) Alerta con **link al log/trace** · 4) Solo lo que alguien puede arreglar.

```
ALERTA "Error rate GET /api/feed > 5% durante 5 min"
ACCIÓN: revisar Edge Function / query de feed / saturación de DB
```

Resiliencia (self-healing)

Técnica	Qué hace
Timeouts	Ningún request se cuelga para siempre
Retries + backoff	Reintenta esperando más cada vez (1s, 2s, 4s...)
Circuit breaker	Tras N fallos, falla rápido en vez de pegarle al servicio caído
Fallbacks	Si la fuente falla, sirve cache / datos locales

En Supabase los logs de Postgres, Auth, Realtime, Storage y Edge Functions ya son gestionados (dashboard + API). No montes un stack de observabilidad hasta que el tráfico lo justifique.

Los 4 niveles

Nivel	Qué guarda
1. Lectura offline	Cache de lo ya visto (feed cargado)
2. Escrituras offline	Acciones en cola local (like, mensaje)
3. Sync en background	La cola se envía al reconectar
4. Conflict resolution	Ambos lados editaron lo mismo

Estrategia recomendada: optimistic UI + cola

1. Usuario da like → cola local + UI actualiza al instante

2. Sin red: queda "pending" en la cola

3. Al reconectar: se envía en orden con retries + backoff

4. Éxito → "synced" · fallo → reintento con límite

Conflictos: definirlos ANTES

Política	Qué pasa
Last-write-wins (LWW)	Gana el último <code>updated_at</code> . Simple; pierde trabajo.
Merge manual	Se conservan ambos y el usuario decide
Por campo	El último cambio por cada campo

Por qué funciona con Supabase: Realtime empuja cambios remotos · RLS decide qué sincronizar · **UUIDs como PK** (sin colisiones de sync) · `updated_at` para LWW y delta sync. Diseño AP: disponible siempre, consistente eventualmente.

1 • Feed de Red Social (read-heavy)

Decisión	Trade-off
Cache-aside del feed (Redis)	Primer acceso lento tras expiración
Postgres como fuente de verdad	Consistencia fuerte; escala menor
CDN para media	TTL/invalidación al borrar
Cache local Isar (offline)	Sync = complejidad extra

CAP: AP (una foto "unos segundos tarde" no es crítico); el post en sí usa strong consistency para no perder publicaciones. Escala: ~1.7K QPS lectura, 10:1.

2 • Chat / Mensajería (write-heavy + realtime)

Decisión	Trade-off
Realtime (WebSocket) para entregar	Gestión de conexiones, límites
Insert en Postgres como "canal"	Realtime depende de la DB
Cola + optimistic UI	Complejidad de sincronización
UUIDs + LWW por <code>updated_at</code>	IDs más largos

CAP: AP en la entrega percibida, pero el mensaje **nunca se pierde**: persistencia fuerte en el servidor. RLS crítico: leer solo si participas en la conversación.

3 • E-commerce Escalable (dinero + stock)

Decisión	Trade-off
Catálogo con cache-aside	Stock visible puede estar desfasado
Checkout transaccional (ACID)	Checkout más lento que el resto
Edge Function para checkout	Complejidad de despliegue
Idempotencia en pagos	—

```
// Evita el oversell (2 compradores, 1 unidad):
BEGIN;
SELECT stock FROM products WHERE id = $1 FOR UPDATE; -- lock
-- stock >= qty ? resta + inserta order : error "sin stock"
COMMIT;
```

CAP: CP en checkout (prefiere rechazar antes que dejar pasar un oversell); catálogo AP.

4 • SaaS Multi-Tenant (aislamiento)

Decisión	Trade-off
Single Postgres + RLS como aislador	Políticas complejas; riesgo si una falla
Roles por membresía (admin/member/viewer)	Más SQL de policies
Cache namespaced por <code>org_id</code>	Invalidar por tenant al escribir
Test de aislamiento en CI	Suite dedicada

```
CREATE POLICY "read_projects_of_my_orgs" ON projects
FOR SELECT USING (EXISTS (
  SELECT 1 FROM organization_members m
  WHERE m.org_id = projects.org_id AND m.user_id = auth.uid()
));
```

CAP: CP — la consistencia de datos de proyectos importa más que servir data eventual. Riesgo clave: **noisy neighbor** (un tenant saturado) → cuotas y rate limits.

Fase	Error típico	Solución
Alcance	Diseñar sin clarificar · olvidar el fuera de scope	Preguntar usuarios/carga/latencia ANTES de dibujar
Escala	Sin números · ratios ignorados	QPS, storage y ratio L/E siempre
Datos	NoSQL "por moda" · indexar todo	RDBMS primero · índices solo para path reales
Cache	Sin TTL/invalidación · ignorar el stampede	TTL + jitter + invalidación por escritura
Realtime	WebSocket para todo · solo REST cuando piden realtime	Árbol de decisión: polling → SSE → WS
CAP	No declararlo · consistencia fuerte en todo	Elegir CP o AP por dato y justificar
Seguridad	Service role en la app · solo login sin RLS	Menor privilegio + RLS deny-by-default
Observabilidad	Métricas sin logs/traces · alertas por todo	Los 3 pilares + alertas accionables
Offline	Sin política de conflictos · IDs autoincrementales	LWW definido antes + UUIDs como PK
Cierre	Terminar sin riesgos ni evolución	"El siguiente cuello de botella sería X, lo resuelvo con Y"

Checklist final de un diseño:

alcance + fuera de scope · números justificados · diagrama · decisiones con trade-offs · CAP/SLA declarados · seguridad + observabilidad · riesgos y evolución.

A

Anexo · Índice del módulo (para profundizar)

Necesitas	Archivo
Qué es system design y sus atributos (reliability, scalability, maintainability)	00-fundamentos-system-design.md
Framework de clarificación de requisitos y restricciones	01-clarificar-requisitos.md
Estimación de escala con ejemplos resueltos	02-estimacion-de-escala.md
DNS, CDN, Load Balancer, Reverse Proxy, microservicios	03-componentes-arquitectura.md
Índices, replicación, sharding, denormalización, SQL vs NoSQL	04-modelado-de-datos.md
Estrategias de cache y latencia vs throughput	05-cache-y-rendimiento.md
Polling vs WebSocket vs SSE + Supabase Realtime	06-realtime-y-streaming.md
CAP theorem, scaling, disponibilidad en "nueves"	07-escalabilidad-alta-disponibilidad.md
RLS, authn vs authz, menor privilegio, defensa en profundidad	08-seguridad-en-diseno.md
Logs, métricas, tracing, alertas, resiliencia	09-observabilidad-monitoreo.md
Offline-first, sincronización y conflictos	10-offline-first.md
Plantilla de 10 pasos reutilizable	11-plantilla-diseno-sistema.md
Caso resuelto: feed de red social	12-caso-feed-red-social.md
Caso resuelto: chat / mensajería	13-caso-chat-mensajeria.md
Caso resuelto: e-commerce escalable	14-caso-ecommerce-escalable.md
Caso resuelto: SaaS multi-tenant	15-caso-saas-multi-tenant.md
Ejercicios progresivos + auto-entrevista	16-ejercicios-practica.md
Libros, guías y recursos verificados	17-recursos-externos.md
Fuentes que fundamentan el módulo	BIBLIOGRAFIA.md

System design en una línea: Alcance → Escala → Datos → Arquitectura → Componentes → Decisiones (trade-offs) → CAP/SLA → Seguridad → Observabilidad → Riesgos. · Regla de oro: si cada decisión tiene su trade-off y conoces tu CAP y tu ratio L/E, ya tienes medio diseño.